SESSION 21 • DEVOPS CAPSTONE PROJECT

StudySpace

From student planner to monitored cloud deployment

Application / Containers / CI/CD / Kubernetes / Observability

AWS demo completed in ap-southeast-2
Resources destroyed after evidence capture

Tanush Shoor • 24BCS10265

PROJECT AT A GLANCE

One app, delivered through a complete DevOps workflow

The capstone connects product behavior with repeatable delivery, deployment, and monitoring.

1 Build the product

React dashboard • FastAPI • PostgreSQL
Alembic schema migration

2 Prove and package

Six API tests pass • Docker Compose
Non-root runtime containers

3 Secure delivery

GitHub Actions • Trivy scans
SHA-tagged GHCR images

4 Deploy

Terraform VPC and EKS
Helm chart • Kubernetes Ingress

5 Observe

Prometheus scrape targets UP
Grafana API dashboard

6 Clean up

58 Terraform resources destroyed
AWS cluster confirmed absent

APPLICATION ARCHITECTURE

A small full-stack app with clear service boundaries

The same frontend and API run locally in Compose and as services in Kubernetes.

Student browser
localhost:3000
or cloud Ingress

HTTPS

React + Vite
Dashboard and forms
NGINX unprivileged image

/api

FastAPI
REST API • /health • /ready
/metrics • Prometheus

SQL

PostgreSQL
Assignment storage
Alembic migrations

Local development

Docker Compose starts frontend, backend, and database with health checks.

Cloud runtime

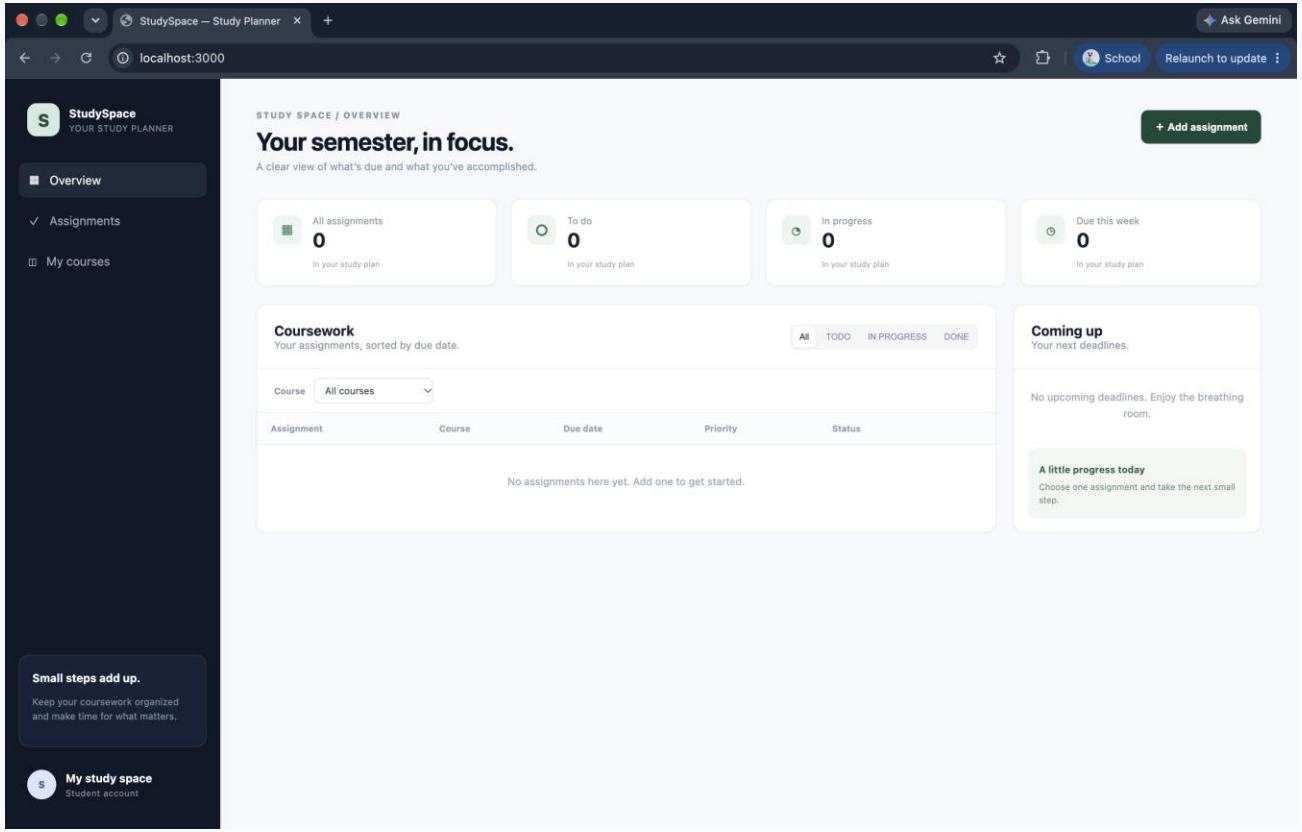
EKS deployments and Services run the same app; Ingress exposes the frontend.

Metrics flow from FastAPI → Prometheus → Grafana

APPLICATION

The product: StudySpace planner

The React dashboard gives students one place to track assignments, due dates, status, and priority.

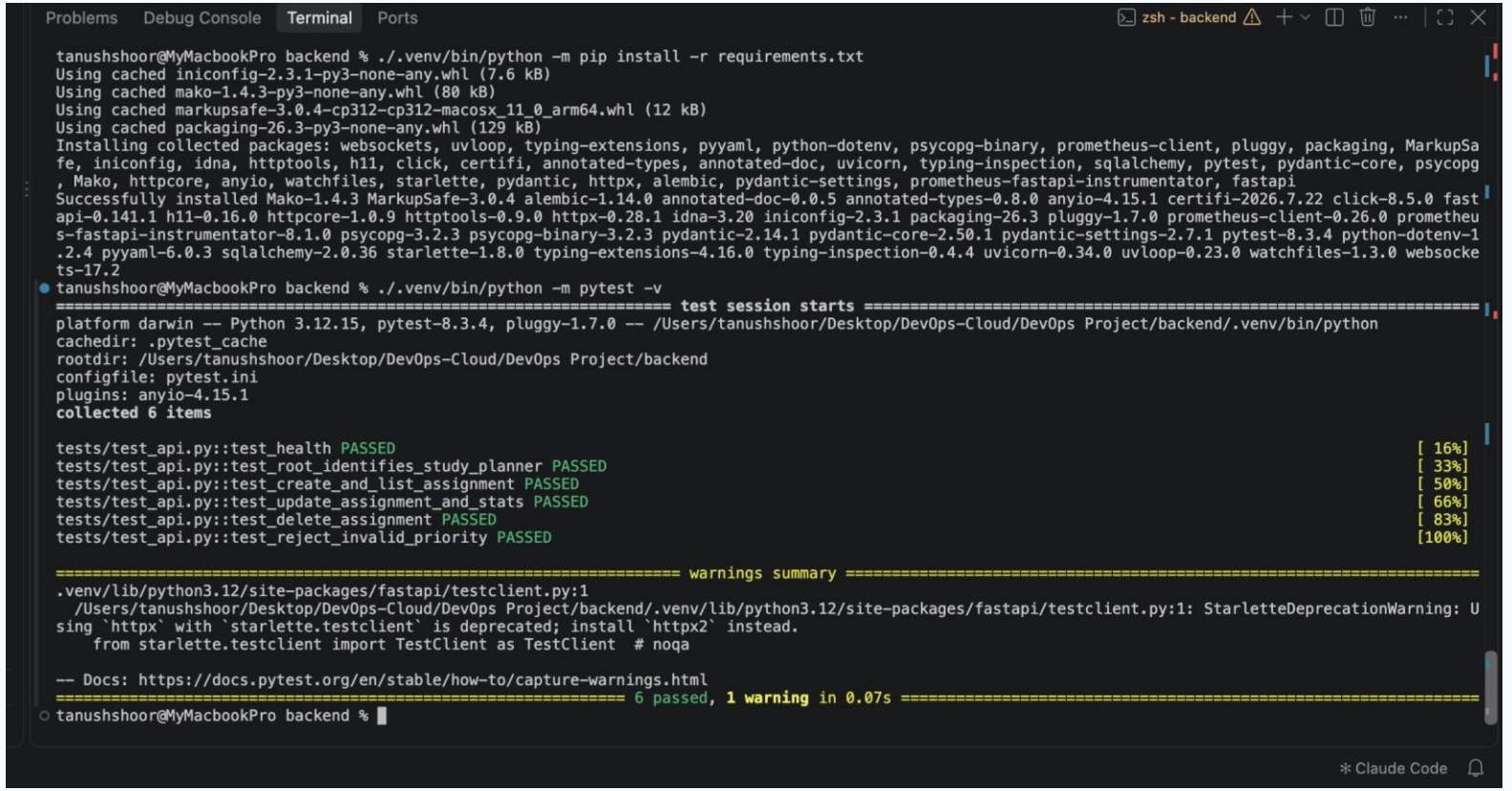


The captured view is the initial empty state; the UI is running locally at port 3000.

TESTING

Automated API tests: 6 passed

The test suite covers health, root response, assignment CRUD, statistics, and validation.



Run shown with Python 3.12.15. One Starlette/httpx deprecation warning is non-failing.

Docker Compose brings the stack up

Compose builds and starts the frontend, backend, and PostgreSQL with service health checks.



Local entry points: frontend :3000 • API :8000 • PostgreSQL :5432.

Three services are running together

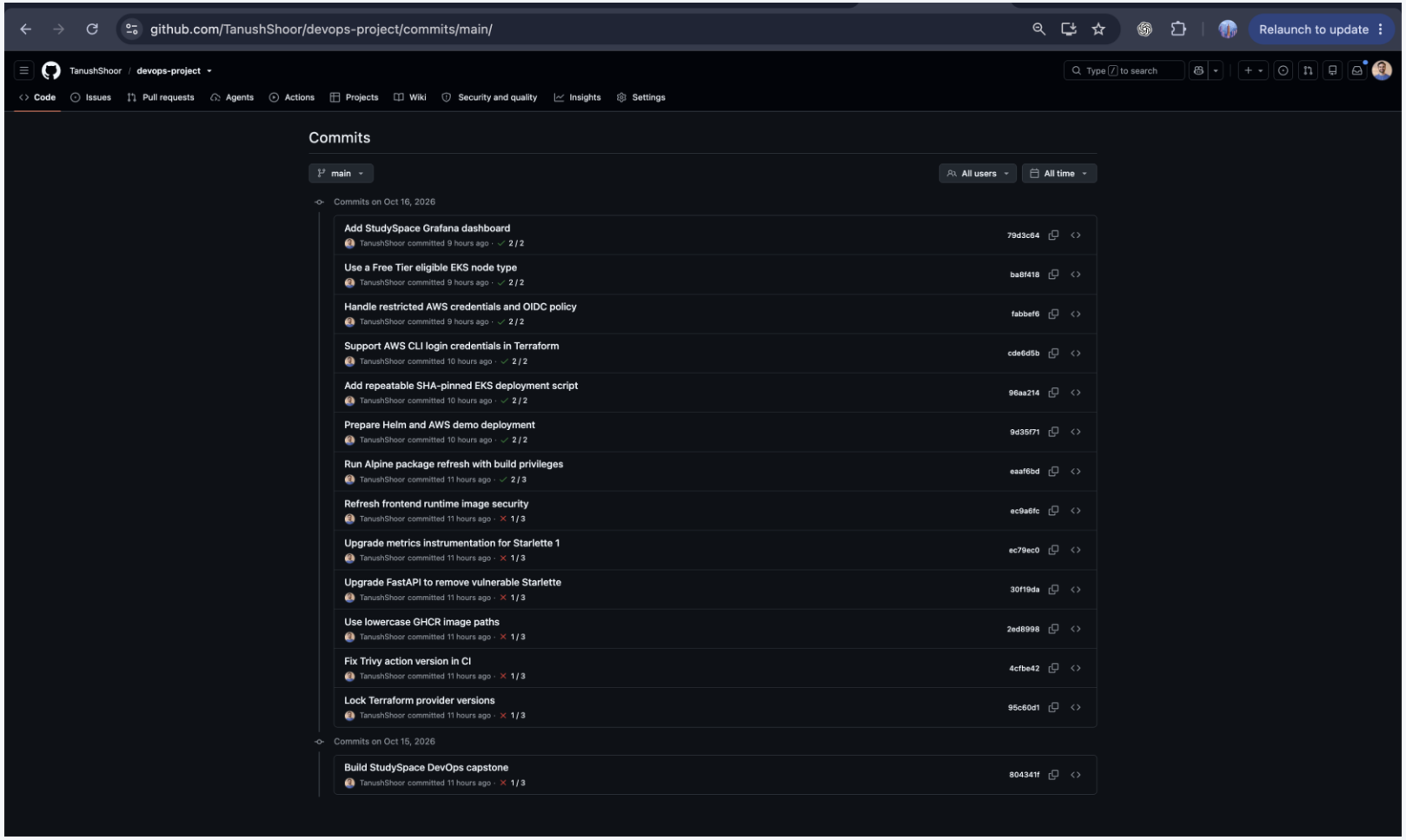
The container view confirms the app components are separated and communicating.

```
tanushshoor@MyMacbookPro DevOps Project %  source "/Users/tanushshoor/Desktop/DevOps-Cloud/DevOps Project/backend/.venv/bin/activate"
((.venv) ) tanushshoor@MyMacbookPro DevOps Project % docker compose ps
NAME                      IMAGE                                                                  COMMAND                  SERVICE    CREATED       STATUS                 PORTS
devopsproject-backend-1   sha256:041ef2719e8ff03338b2b72aaf0db962a5b8fea8e01403023988e4aee677cf50   "sh -c 'alembic upgr…"   backend    11 hours ago   Up 11 hours (healthy)   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
devopsproject-frontend-1  sha256:782b2ea57ef580973bcb6e4f80271d9ea459a350e69927f8ecf2f099682003fe   "/docker-entrypoint.…"   frontend   11 hours ago   Up 11 hours             0.0.0.0:3000->8080/tcp, [::]:3000->8080/tcp
devopsproject-postgres-1  postgres:16-alpine                                                     "docker-entrypoint.s…"   postgres   11 hours ago   Up 11 hours (healthy)   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
((.venv) ) tanushshoor@MyMacbookPro DevOps Project %
```

This is the reproducible local demo path used before the AWS deployment.

Git history records incremental delivery

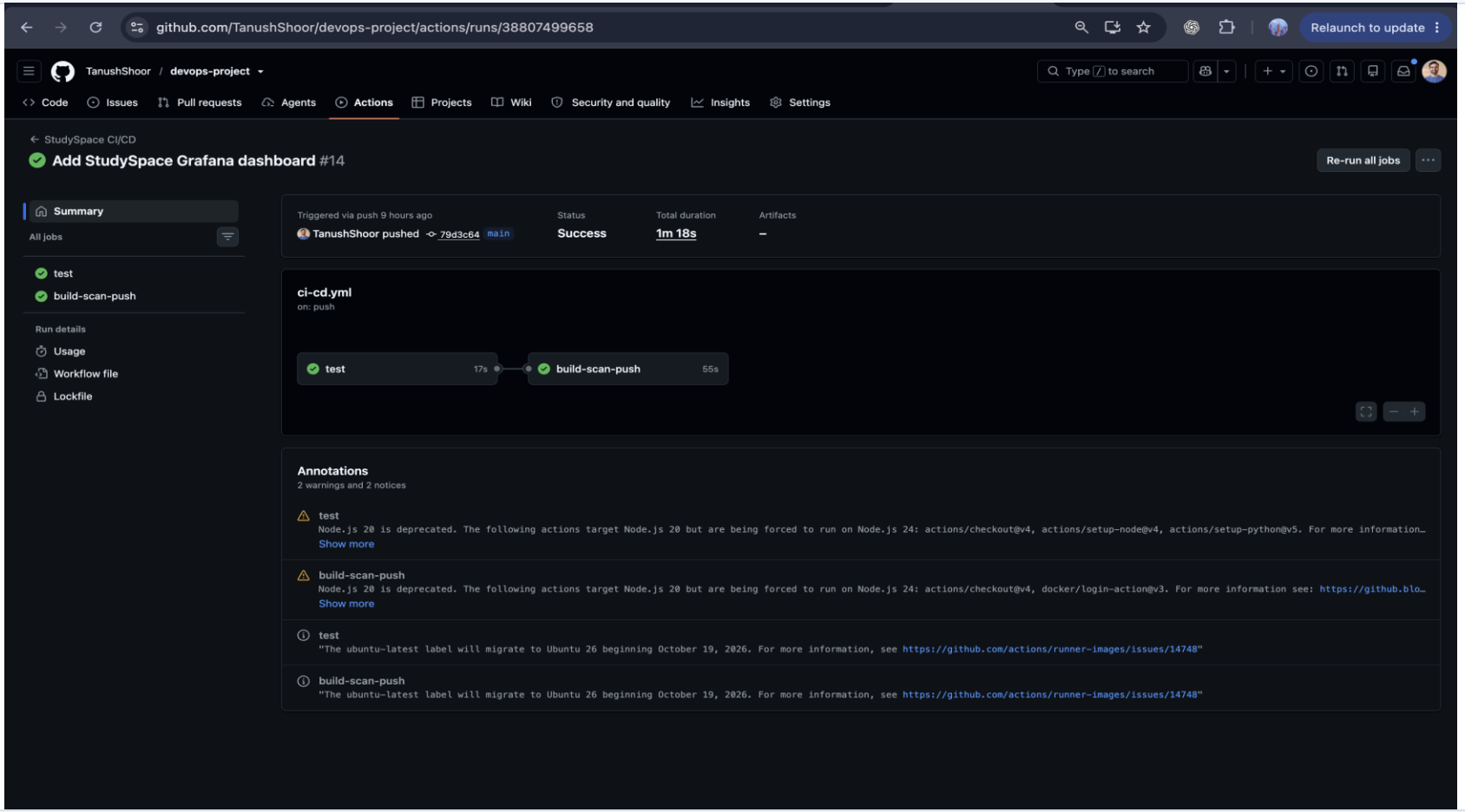
The repository history shows the project evolving through reviewable changes.



Source repository: github.com/TanushShoor/devops-project.

GitHub Actions: final pipeline succeeds

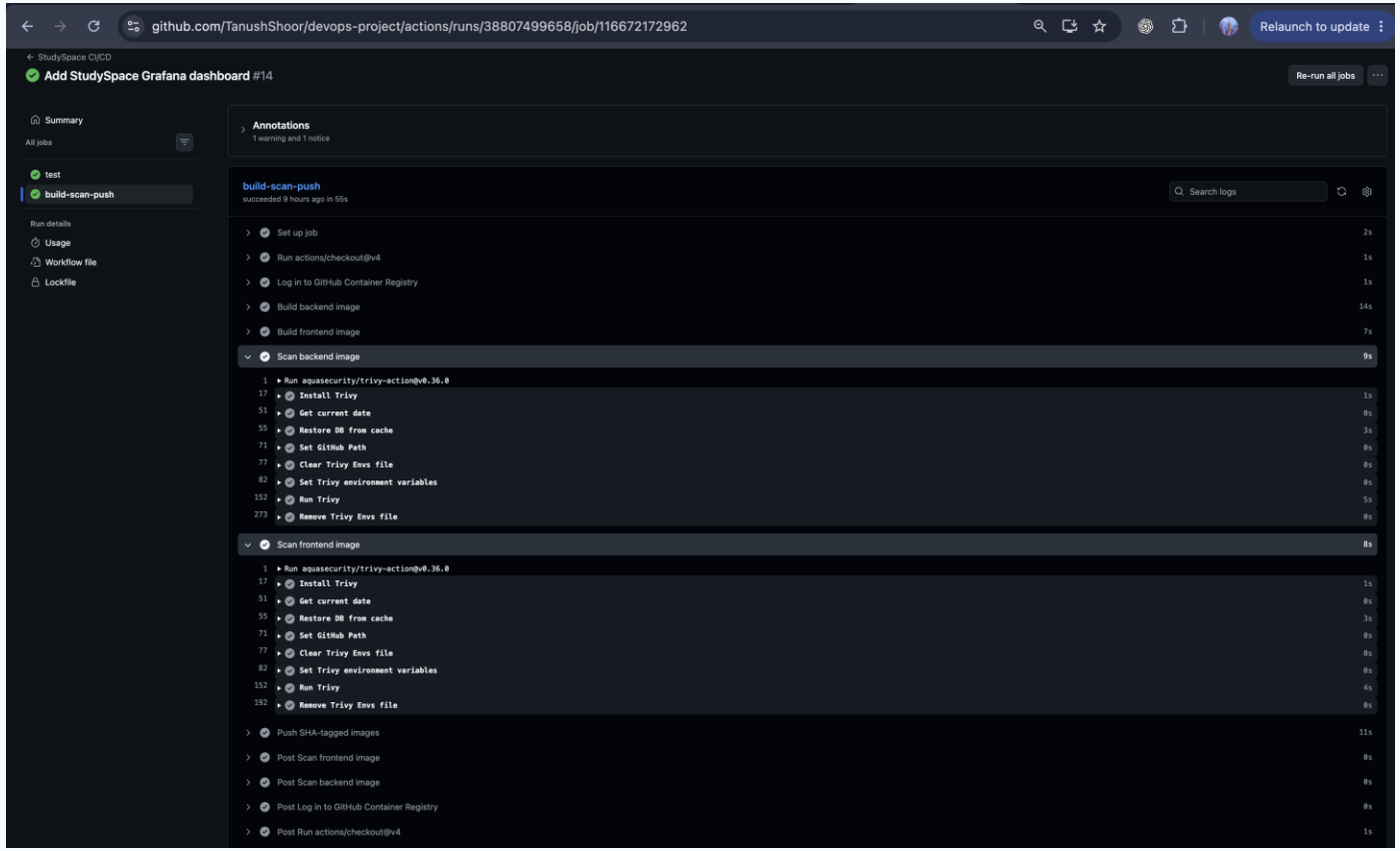
The final captured run is green for the tested commit and build/scan/push stages.



Run #14 • commit 79d3c64 • test job and build-scan-push job succeeded.

Security scan stage in the workflow

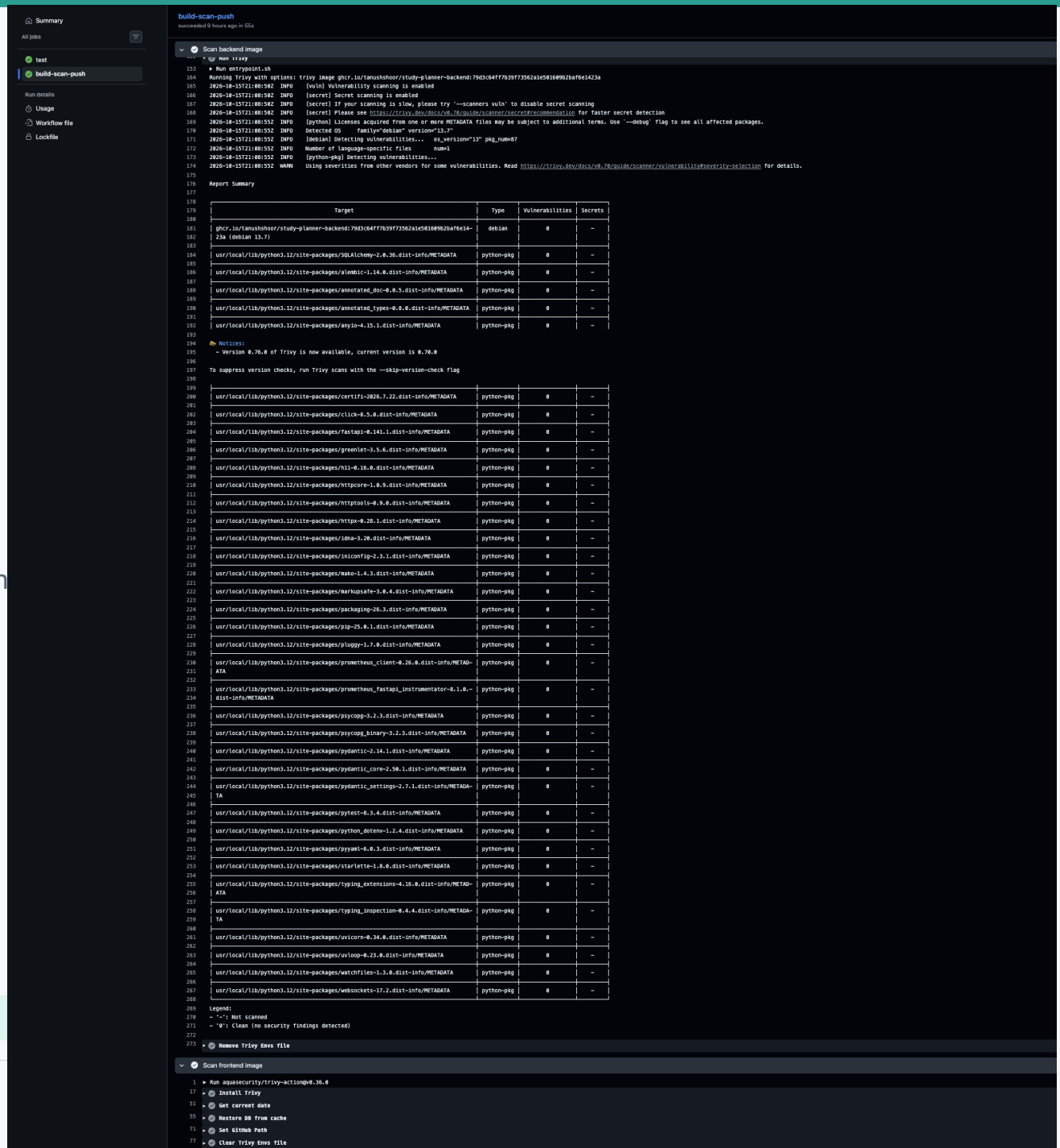
The pipeline scans the built backend and frontend images before publishing.



The workflow blocks images with unfixed HIGH or CRITICAL findings.

Backend image scan result

Trivy reports zero vulnerabilities for the backend image in this captured run



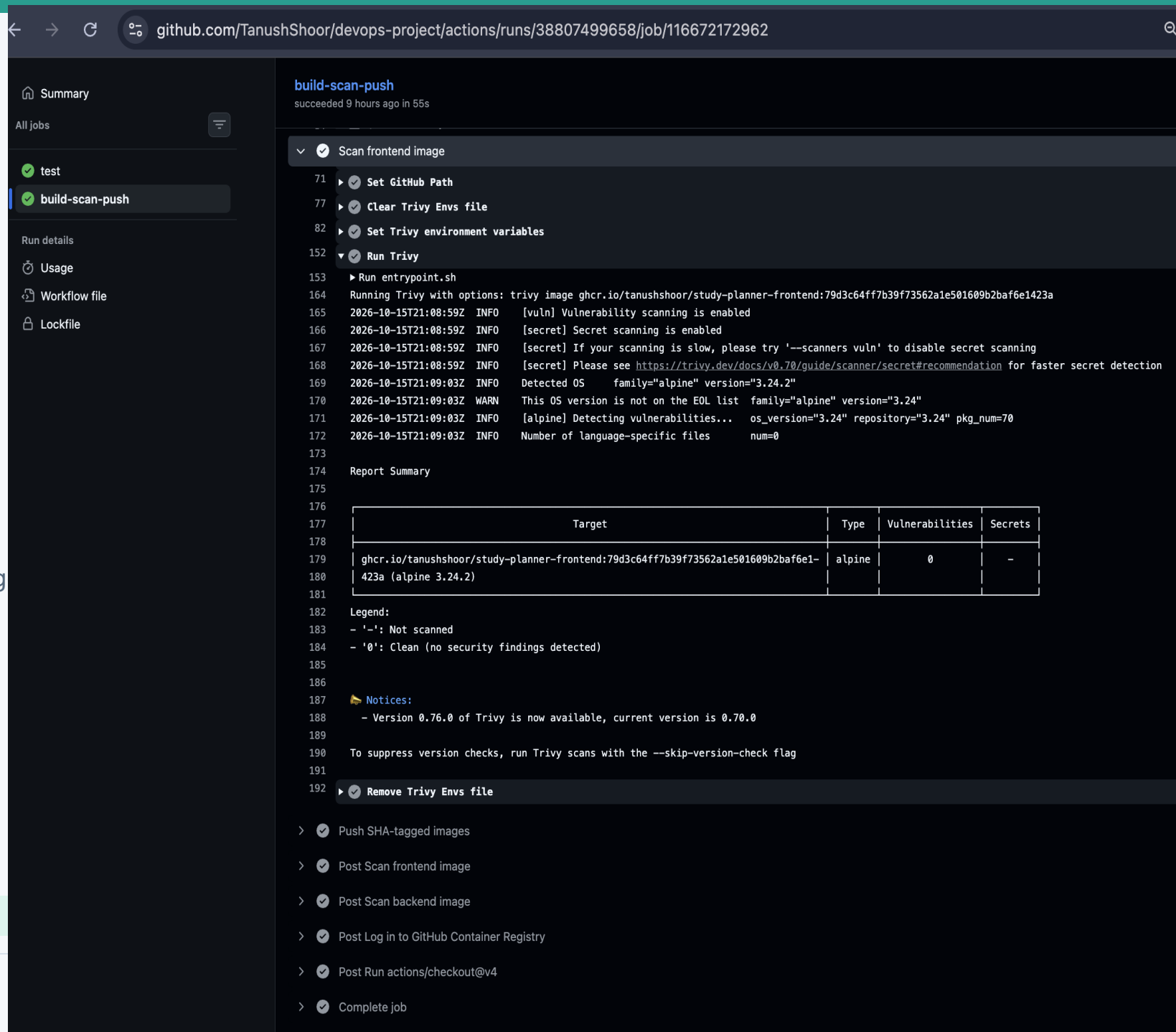
The scan applies to the exact image produced for the recorded commit.

DEVSECOPS

Frontend image scan result

Trivy reports zero vulnerabilities for the frontend imag

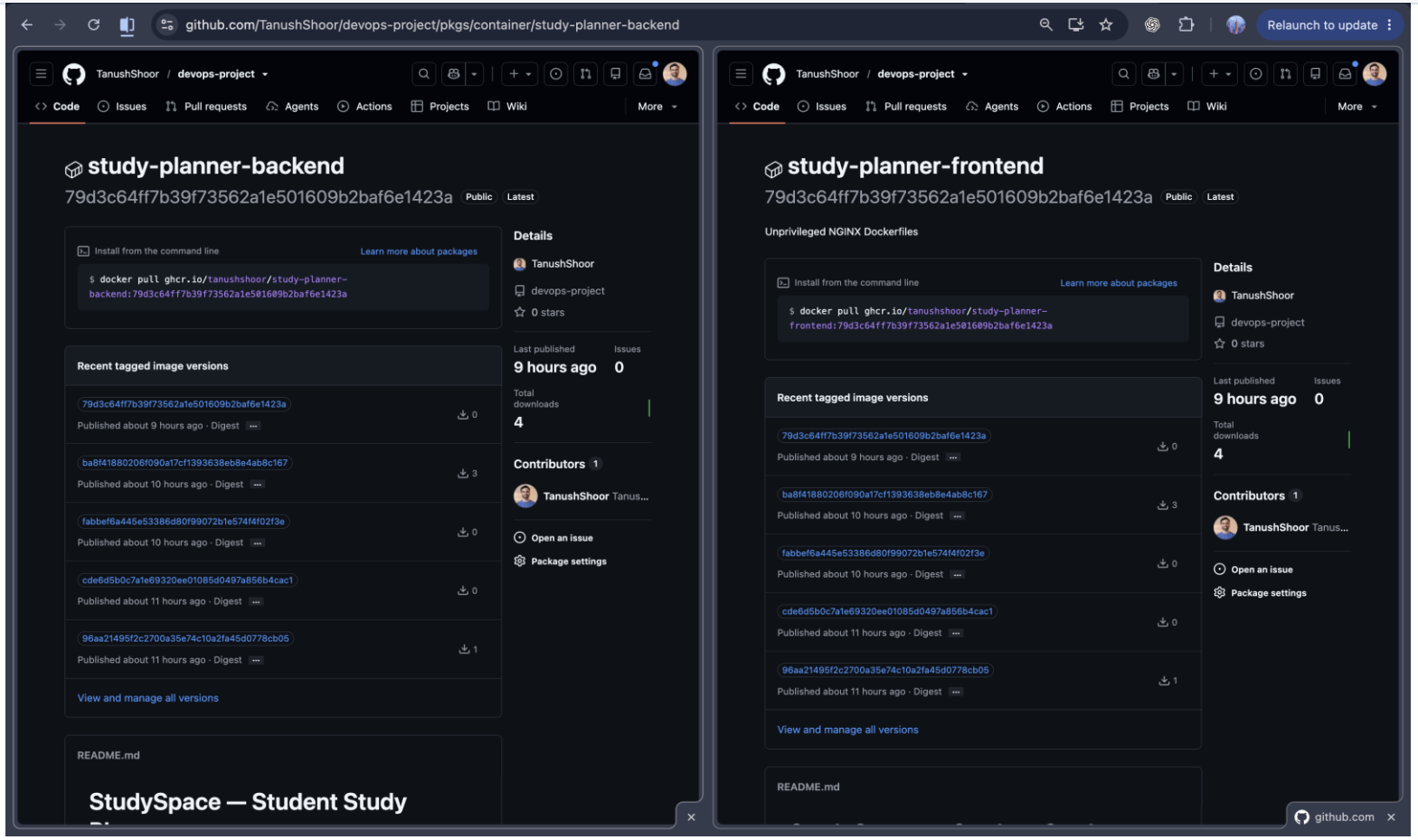
Both application images passed the configured severity gate.



ARTIFACTS

Immutable images published to GHCR

Both services are published under separate package names and tagged with the full commit SHA.

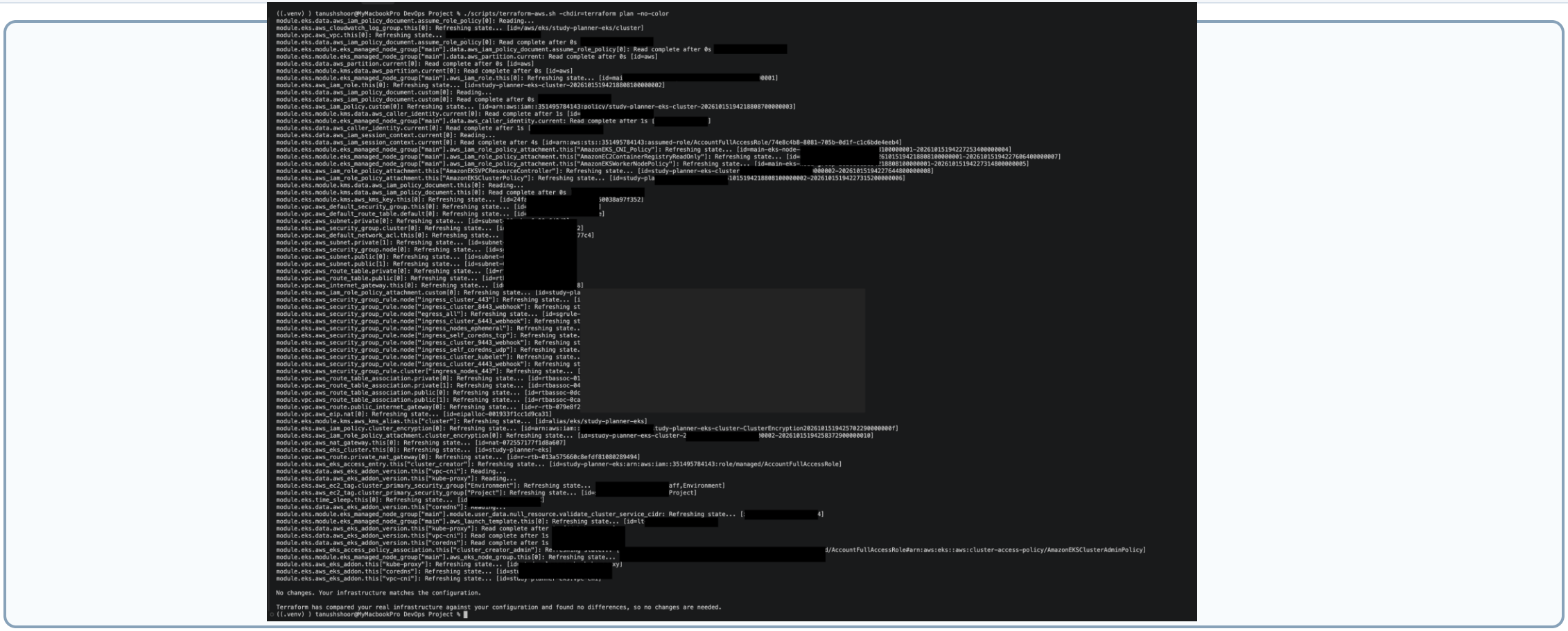


The SHA ties a deployed image back to a specific source revision.

INFRASTRUCTURE

Terraform plan check after provisioning

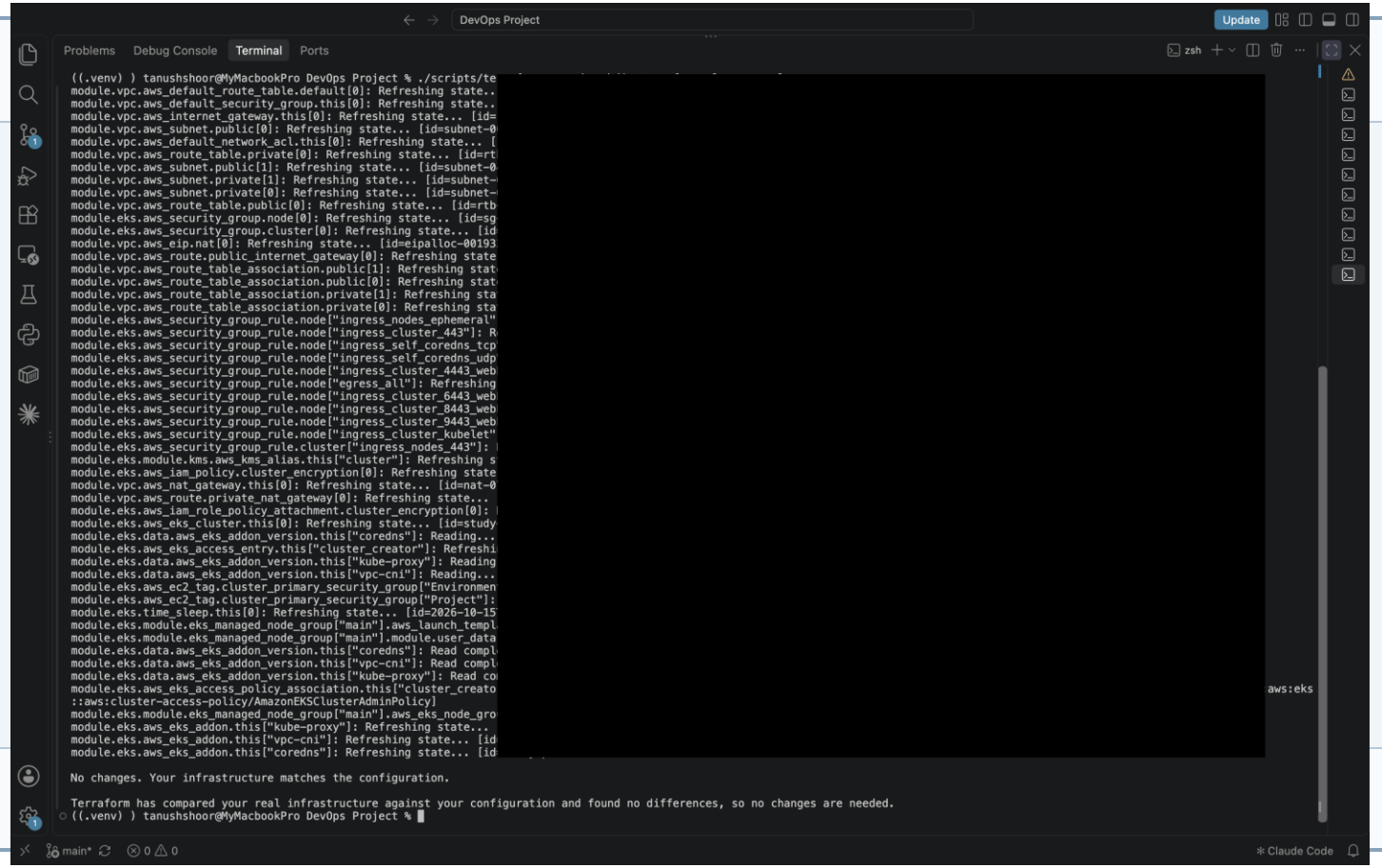
This screenshot is a post-apply plan check and reports no remaining changes.



The original reviewed plan was 4 to add; that initial non-empty plan screenshot is not in the evidence folder.

A second no-change Terraform check

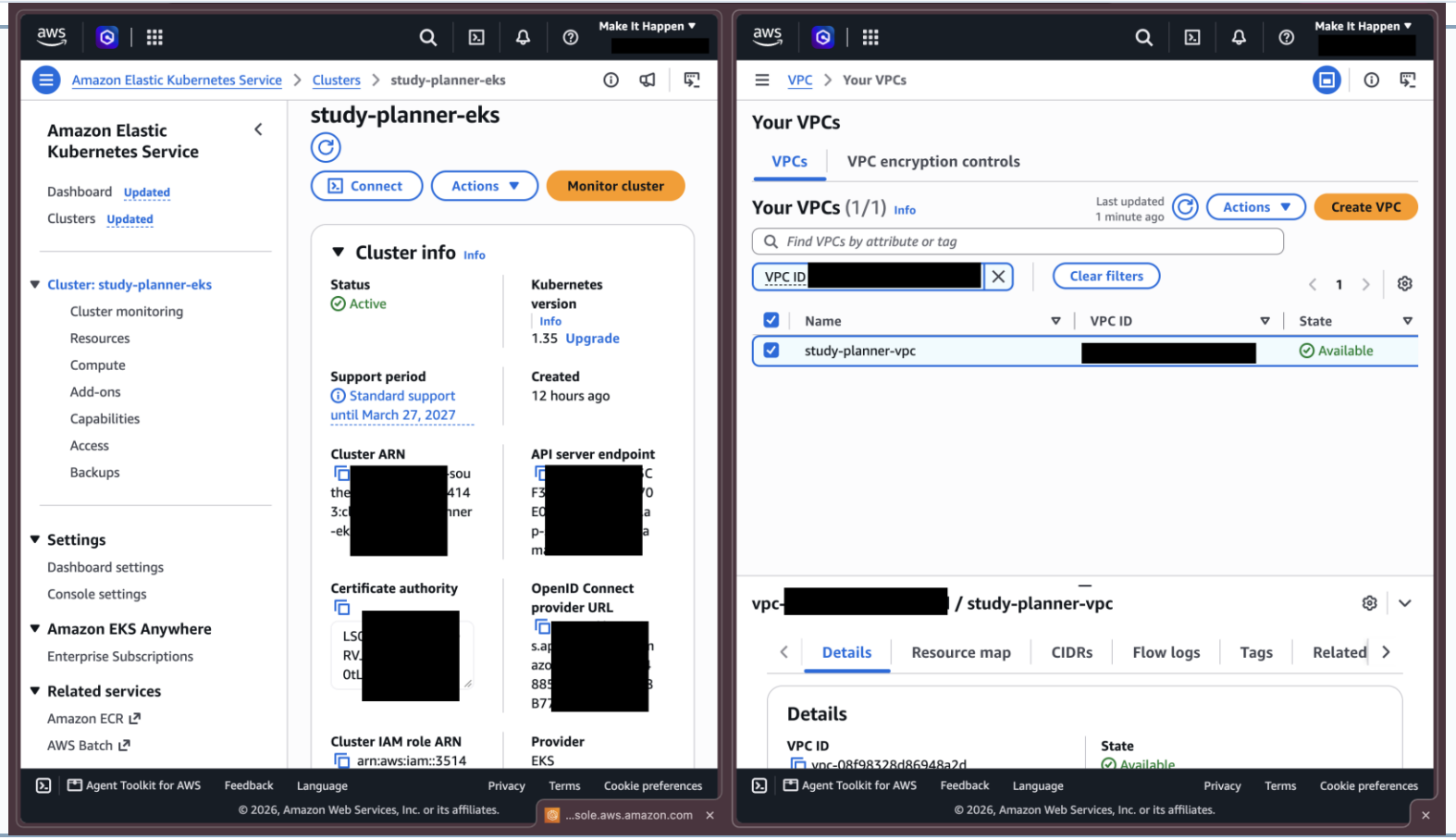
The additional plan capture also confirms the provisioned configuration was stable.



Do not interpret this screenshot as the original create plan; it was taken after deployment.

AWS network and EKS cluster were live

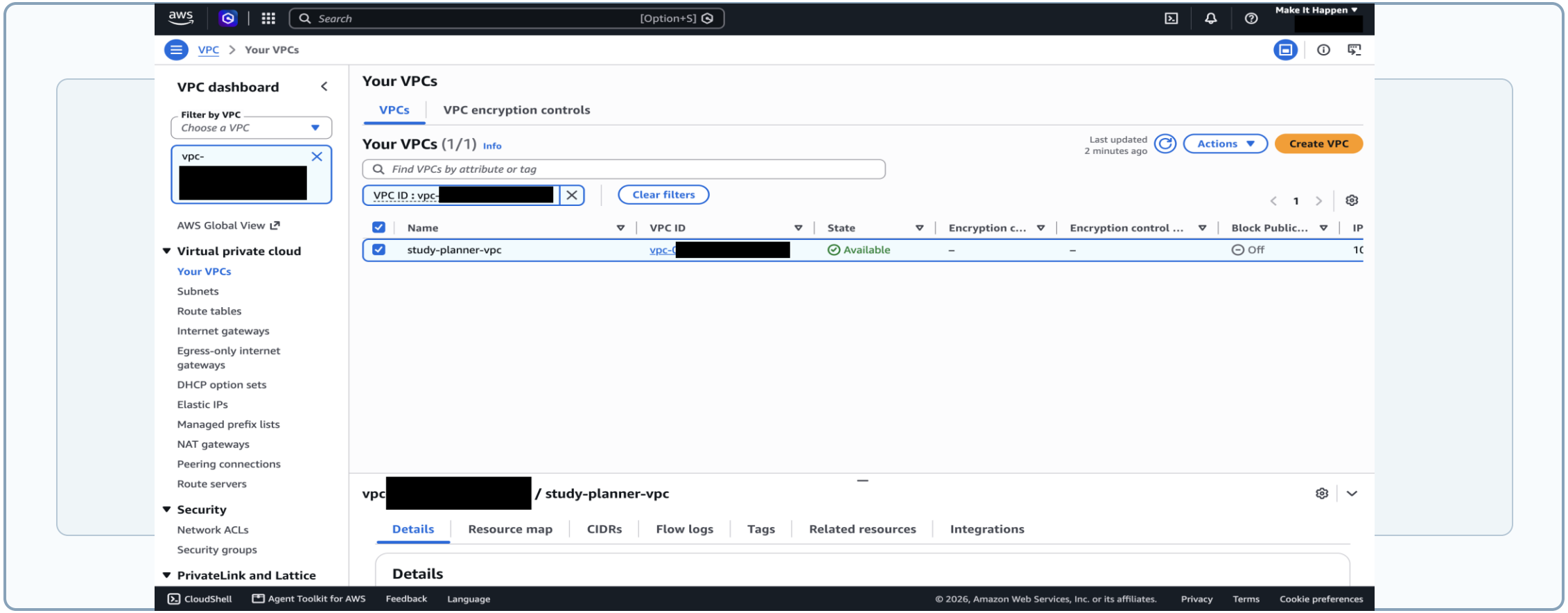
The console evidence shows the VPC and EKS cluster available in ap-southeast-2.



This was a temporary student demo environment and has since been destroyed.

VPC layout: public and private subnets

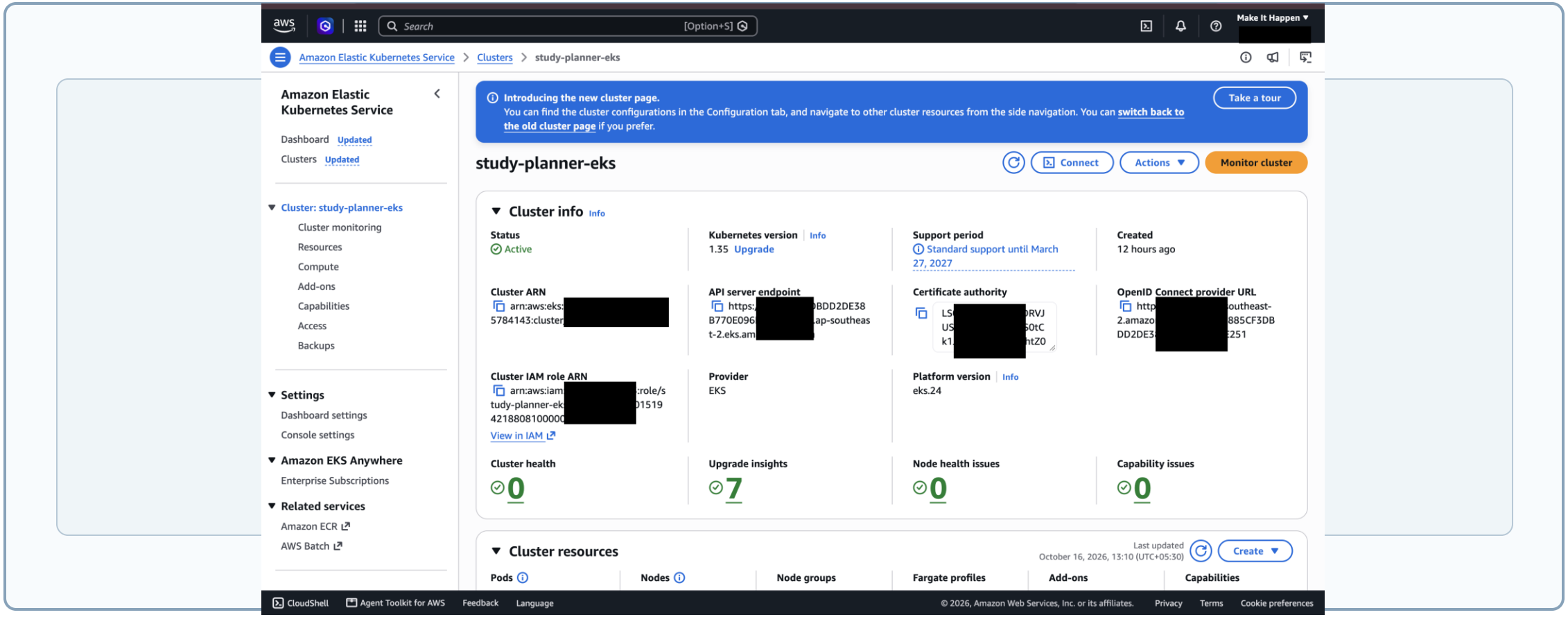
The network layer contains two public and two private subnets with a NAT gateway.



Terraform managed the VPC resources used by the EKS cluster.

EKS control plane and managed nodes

The EKS console confirms the StudySpace cluster and managed capacity during the demo.

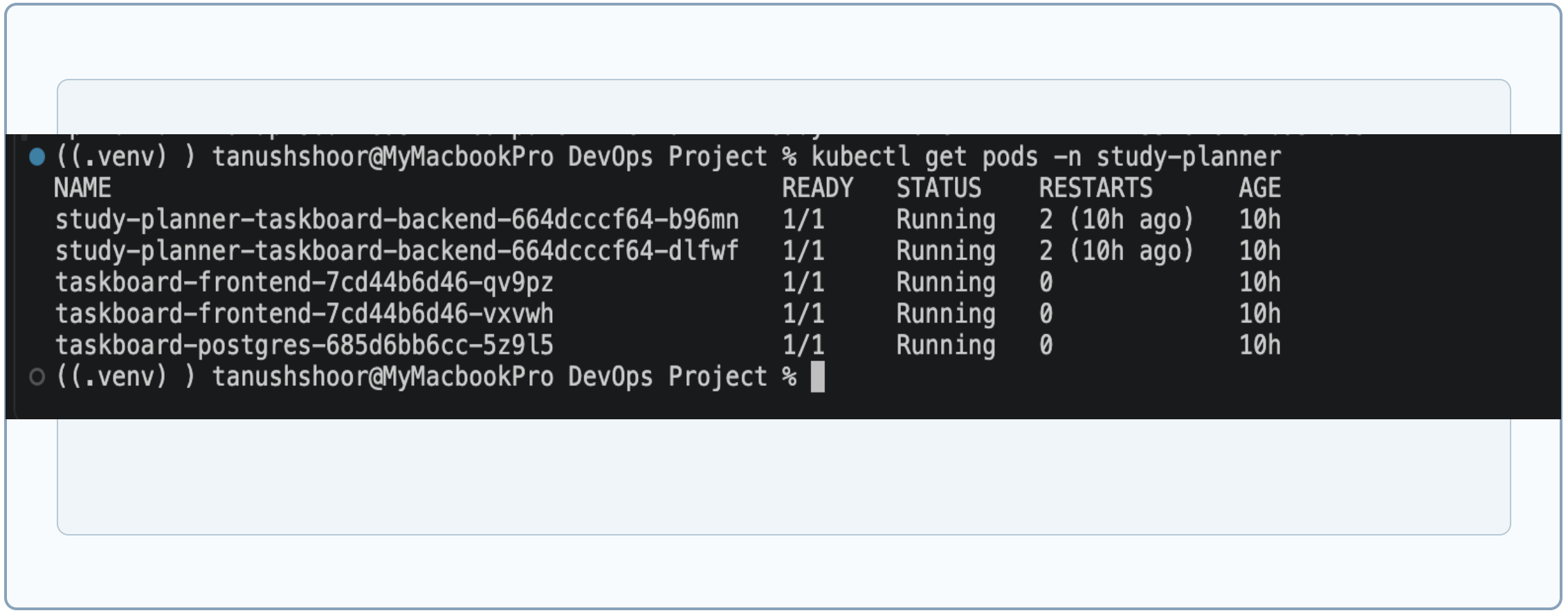


The worker group used two t3.small instances for this short evidence run.

KUBERNETES

Kubernetes workloads reached Running

The application and PostgreSQL pods were scheduled and ready in the cluster.

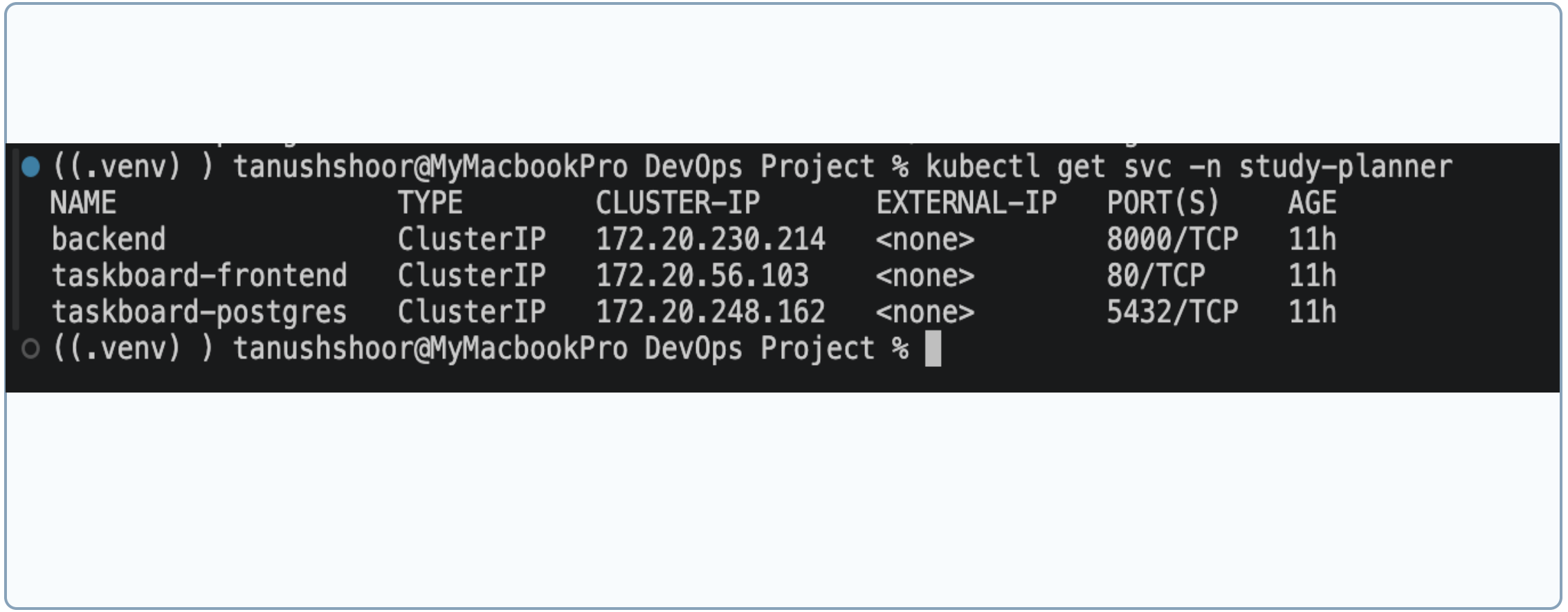


Backend and frontend ran as Deployments with multiple replicas.

KUBERNETES

Kubernetes Services connect the pods

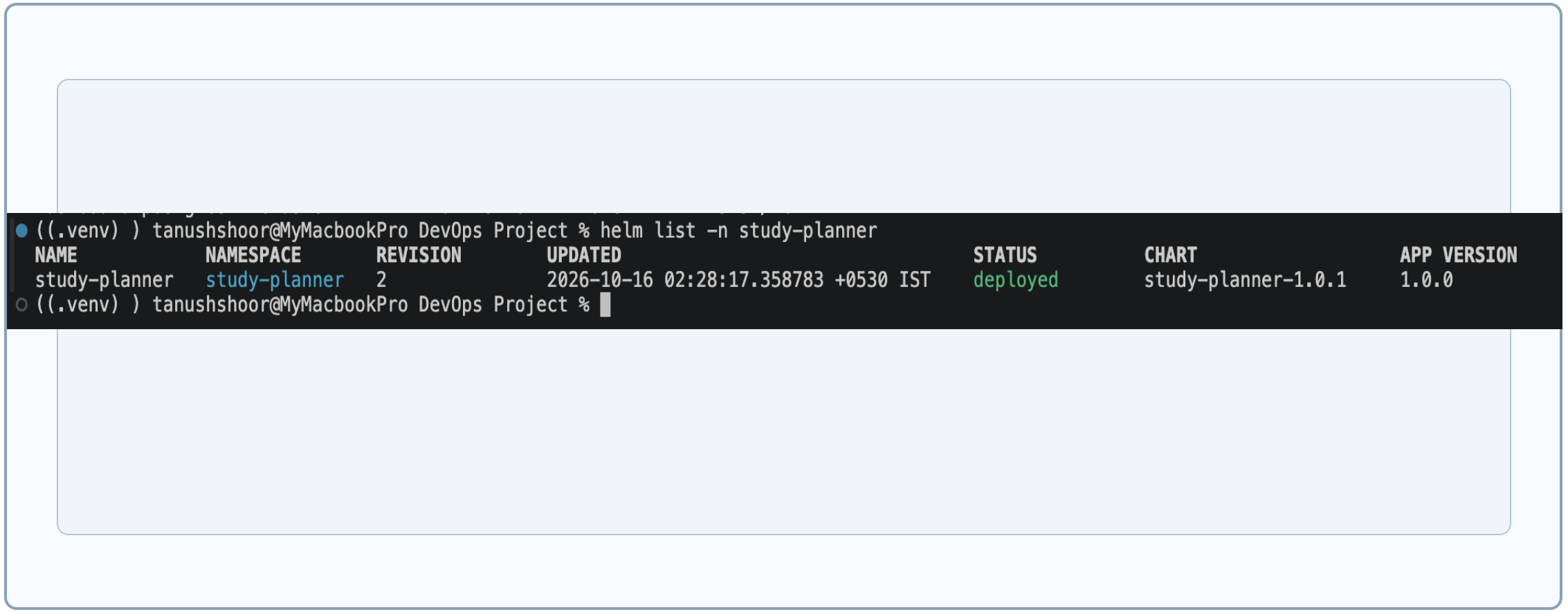
ClusterIP Services provide stable internal endpoints for the frontend, API, and database.



The service layer routes requests independently of individual pod addresses.

Helm manages the StudySpace release

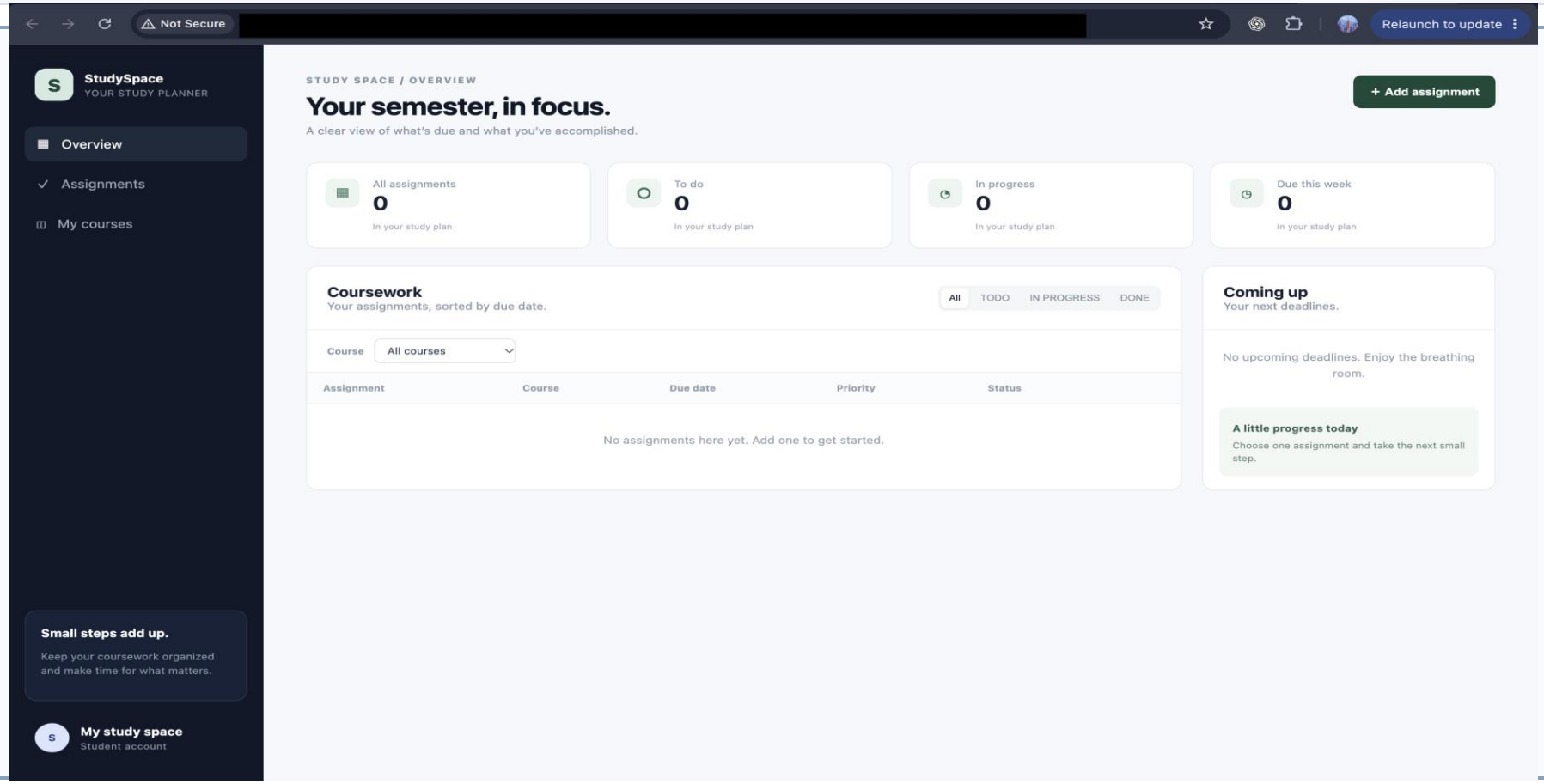
The Helm release records the chart deployment and revision in the cluster.



Helm packages the Kubernetes configuration for repeatable installs and upgrades.

The deployed app was reachable through Ingress

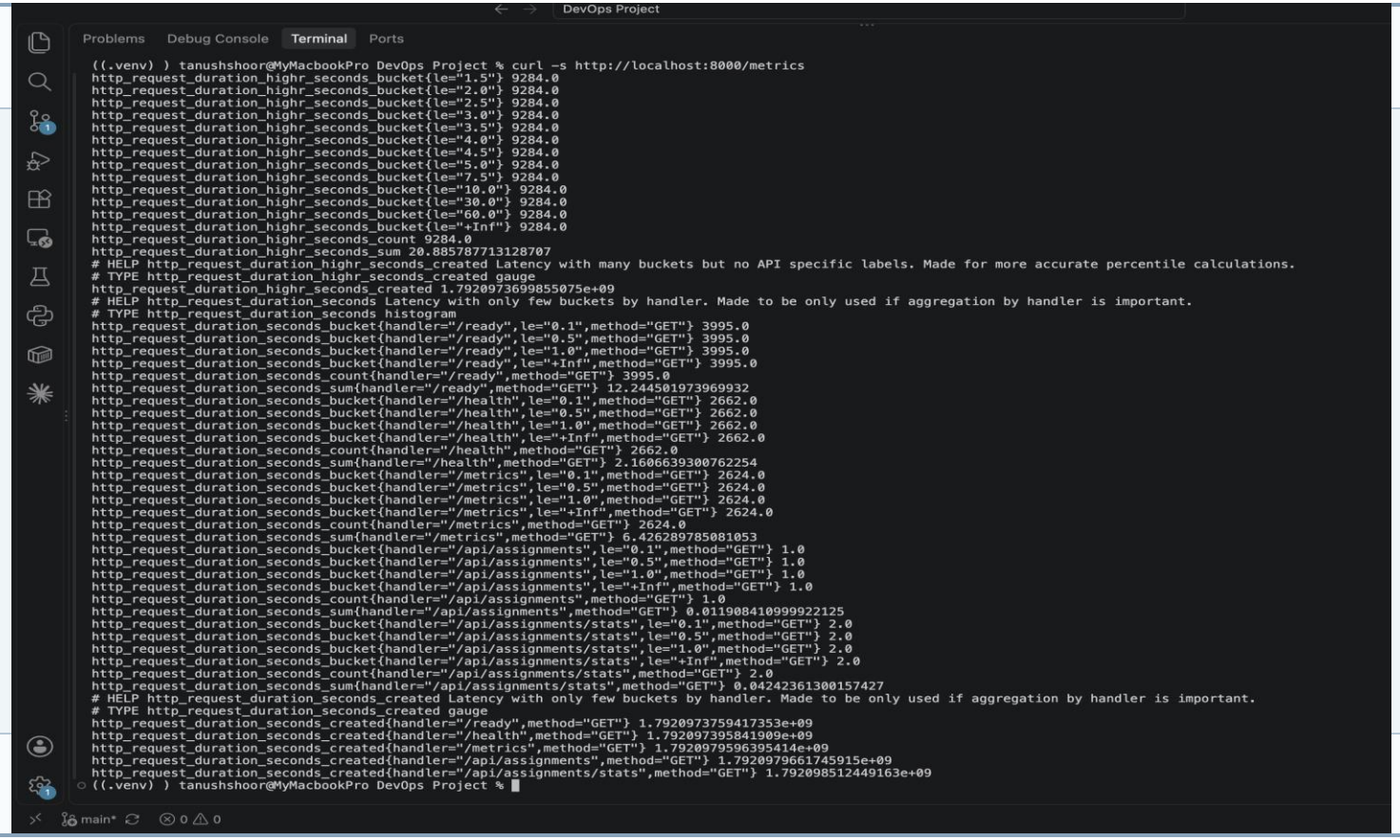
The browser screenshot proves the application was available through the cloud load balancer.



The AWS load balancer endpoint was temporary and is no longer active after teardown.

FastAPI exposes operational metrics

The backend metrics endpoint reports request counts and response measurements.

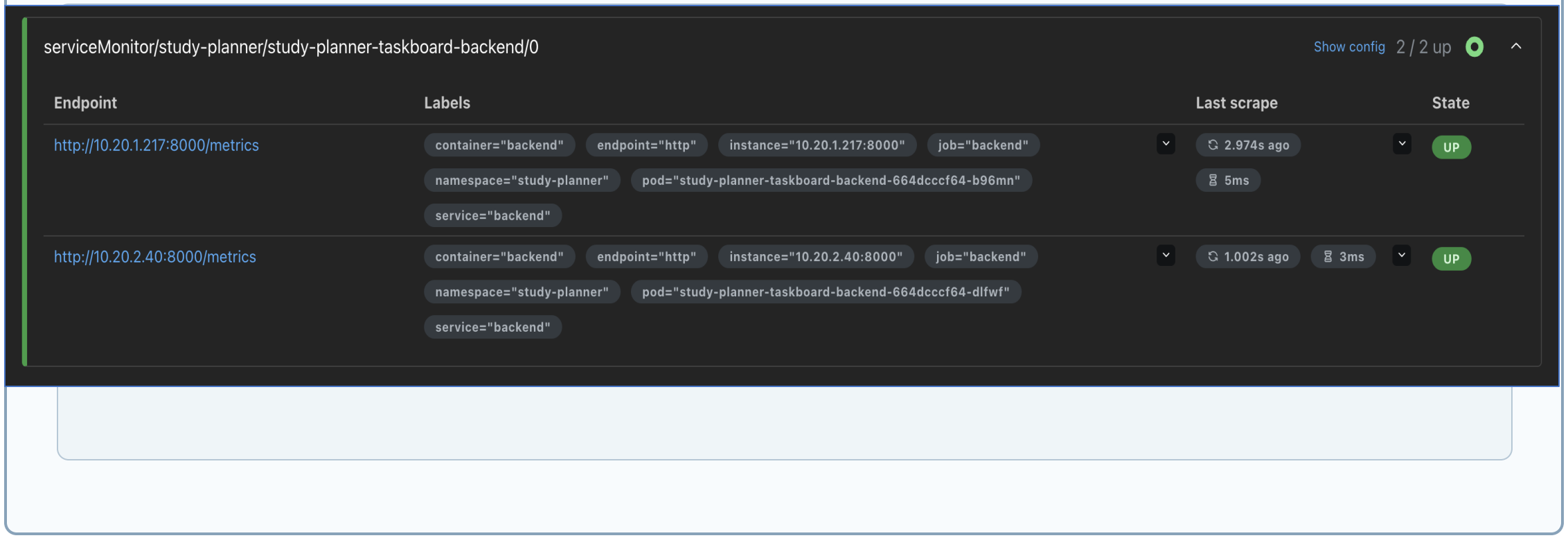


Prometheus can scrape the metrics endpoint for alerting and dashboards.

OBSERVABILITY

Prometheus scraped both backend replicas

Both backend targets are marked UP in the captured Prometheus targets page.

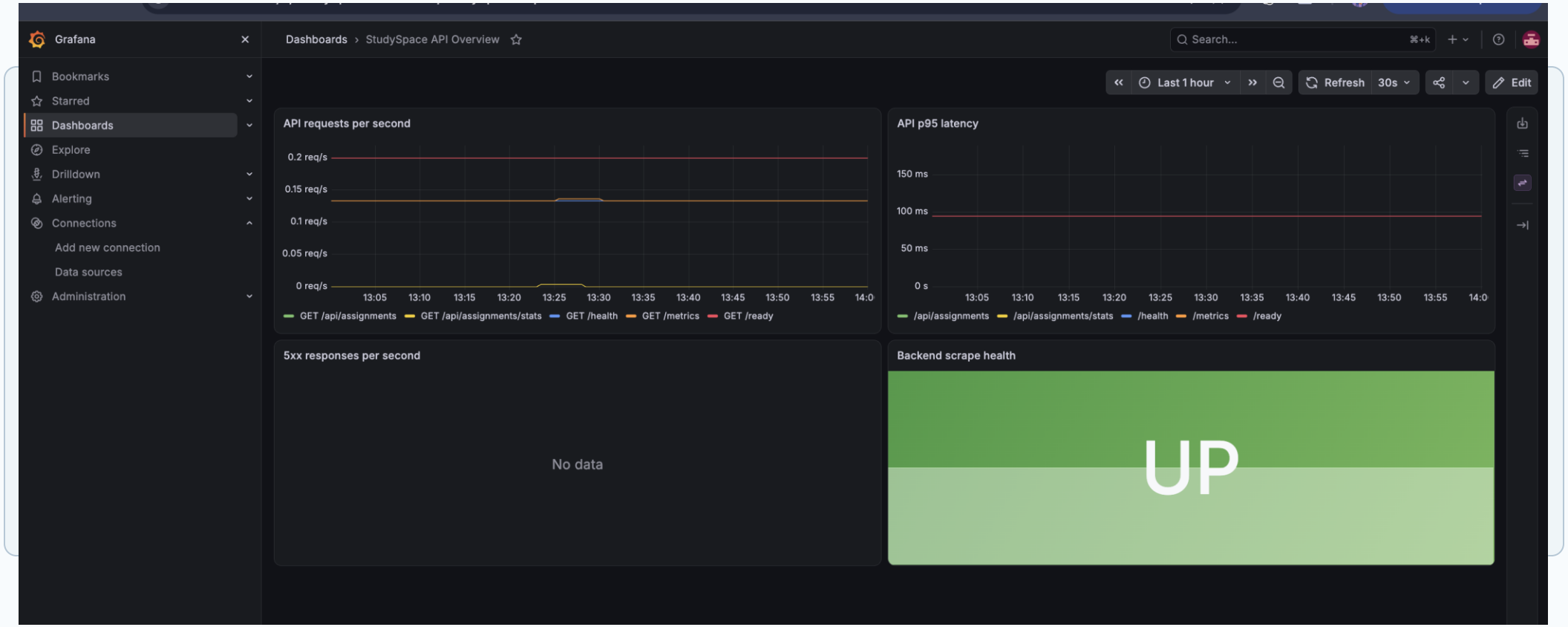


This confirms service discovery and live scraping worked during the demo.

OBSERVABILITY

Grafana visualizes API health

The StudySpace dashboard shows request-rate data and backend scrape health as UP.



The p95 and 5xx panels show no samples in this capture; the dashboard is provisioned for those metrics.

CLEANUP

Cloud resources were destroyed after capture

Terraform reported 58 resources destroyed, and the EKS cluster was verified absent.

```
Destroy complete! Resources: 58 destroyed.
```

18-terraform-destroyed.png

Cleanup keeps the student AWS account from leaving demo infrastructure running.

SUBMISSION SUMMARY

A complete capstone lifecycle, with honest evidence

What is complete, and what a live reviewer should know.

Implemented and evidenced

Application, tests, Compose, GitHub Actions, Trivy, GHCR, Terraform, EKS, Helm, Ingress, and Prometheus/Grafana.

Validated outcomes

6 API tests passed • final CI run green • both Trivy scans clean • Prometheus targets UP • AWS resources destroyed.

Current runtime state

The local Compose stack can be started with `docker compose up --build`. AWS infrastructure is not running.

Live demo sequence

Start Compose for the app; show passing tests and Actions run; use screenshots for cloud stages unless you intentionally reprovision AWS.

Project files: README.md • evidence/ • StudySpace-DevOps-Capstone.pptx